Silsila

Architecture and Class Guide

A knowledge network linking the Qur'an, tafsir, hadith, fiqh issues (mas'ala),
the four madhab positions and modern law (qanun)
for the topics of **nikah** and **talaq**.

Software Engineering course project
Plain Java 22, layered architecture, no frameworks

In this guide

1. What the system does
2. The four layers
3. How a request travels through the layers
4. Folder and file map
5. Every class and what it does
6. How the classes are linked
7. The data model (ids)
8. Requirements coverage
9. Status, limits and what is left

1. What the system does

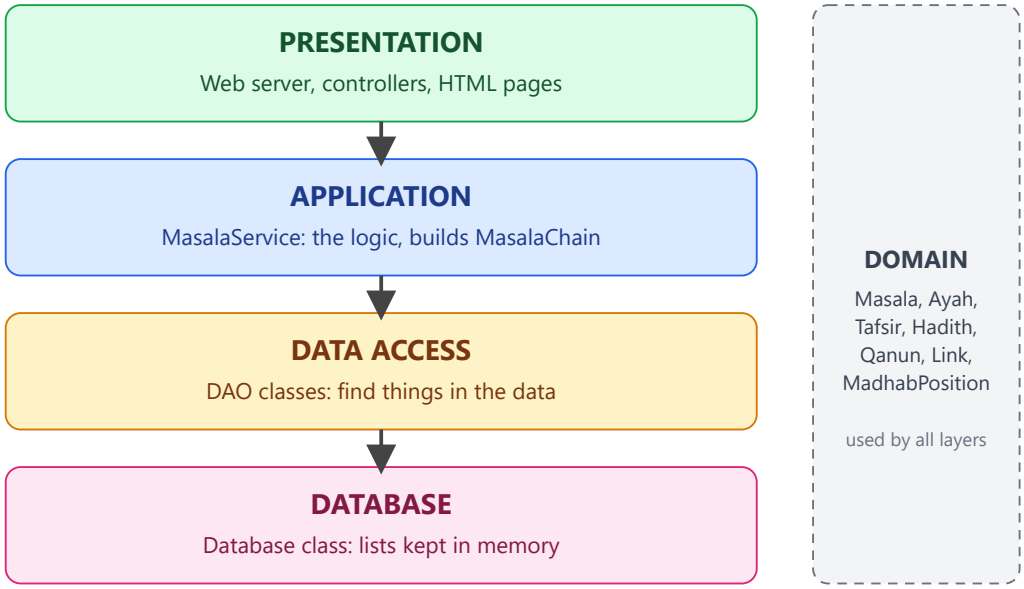
A student opens the website and searches or browses for a fiqh issue, for example *"Is a wali required for the marriage contract?*". Opening the issue shows the whole **chain of evidence**:

Issue (mas'ala) → Verse (ayah) → Tafsir → Hadith → Positions of the four madhabs → Qanun (modern law)

Every piece carries its **citation**, so the student can check the source. Only links that a reviewer has **approved** reach students.

2. The four layers

The program is split into layers. Each layer has one job and only talks to the layer directly below it. This keeps the code easy to understand and to change: for example, changing where the data is stored only touches the data access layer.



Layer	Its job	May talk to
Presentation	Receives the browser's request, asks the service for data, builds the HTML page and sends it back.	Application layer only
Application	Holds the rules and logic: searching, browsing, collecting the whole chain of an issue.	Data access layer only
Data access	The DAO classes. They are the only classes that know where the data lives and how to find it. They also enforce "only approved links reach students".	Database only
Database	Stores the data. Right now it is plain Java lists in memory. Later it can be replaced by a real database without changing the layers above.	Nothing below

Domain	Simple classes that describe the things in our world (an issue, a verse, a hadith...). Every layer uses them to pass data around.	-

3. How a request travels through the layers

Example: the student opens `http://localhost:8080/masala?id=4`.

#	Layer	What happens
1	Browser	Sends the request `/masala?id=4` to the server.
2	Presentation	`HttpServer` sees the address starts with `/masala` and hands it to `MasalaController`. The controller reads `id=4`.
3	Application	The controller calls `masalaService.getChain(4)`. The service asks each DAO for its part.
4	Data access	`MasalaDAO` finds issue 4. `AyahDAO`, `HadithDAO` and `QanunDAO` return only the items whose link is APPROVED. `TafsirDAO` returns the tafsir of those verses. `MadhabPositionDAO` returns the four positions.
5	Database	The DAOs loop through the lists inside `Database`.
6	Application	The service packs everything into one `MasalaChain` and returns it.
7	Presentation	The controller builds the HTML (verses, hadith, four madhab boxes, law), escaping all text, and `Html.send` returns it with status 200. If the id does not exist, it sends a 404 page.

4. Folder and file map

```
Silsila-/
  pom.xml                    Project settings (Java 22)
  README.md                  How to run the project
  docs/                      Requirements (SRD), architecture, diagrams
  src/main/java/com/silsila/
    domain/                  The "things": data classes
      Masala.java  Ayah.java  Tafsir.java  Hadith.java
      Qanun.java  MadhabPosition.java  Link.java
    dataaccess/              Finding data
      Database.java
      MasalaDAO.java  AyahDAO.java  TafsirDAO.java
      HadithDAO.java  QanunDAO.java  MadhabPositionDAO.java
    application/             The logic
      MasalaService.java  MasalaChain.java
    presentation/            The web part
      Main.java  Html.java  Request.java
      SearchController.java  MasalaController.java
```

The folder names are also the Java **packages** (`package com.silsila.domain;` at the top of each file). Packages group classes by layer.

5. Every class and what it does

Domain (7 classes)

Class	What it represents	Main fields / methods
Masala	One fiqh issue, for example "Is there a minimum mahr?".	id, title, category (NIKAH or TALAQ), chapter, description, keywords
Ayah	A verse of the Qur'an. Only its place is stored; the Arabic text is loaded from a website.	id, surahNumber, surahName, ayahNumber, text; `getReference()` gives "Surah Al-Baqarah 2:232"
Tafsir	A scholar's explanation of one verse.	id, ayahId, sourceName, edition, text
Hadith	A hadith. Only the book and number are stored; the text and grade are loaded from a website.	id, collection, collectionName, number, text, grade; `getReference()` gives "Sunan Abi Dawud no. 2085"
Qanun	A modern law article.	id, lawName, article, description; `getReference()`
MadhabPosition	What one madhab says about one issue.	id, masalaId, madhab, ruling, evidence, illah (reasoning), citation
Link	The connection "this issue is supported by this verse / hadith / law". The heart of the project.	id, masalaId, type (AYAH, HADITH, QANUN), targetId, citation, status (PENDING, APPROVED, REJECTED); `isApproved()`

Data access (7 classes)

Class	What it does
Database	Holds seven lists (masalas, ayat, tafsirs, hadiths, qanuns, positions, links) and fills them when the program starts with private methods `addMasalas()`, `addAyat()`, etc. The six issues, five verses, three hadith references, one law, twelve positions and nine links live here.
MasalaDAO	`findAll()`, `findById(id)`, `findByCategory(category)`, `searchByKeyword(text)`. The search lowercases the text, splits it into words and requires every word to appear in the title or keywords.
AyahDAO	`findById(id)`; `findApprovedByMasala(masalaId)` goes through the links of type AYAH that are approved and returns those verses.
TafsirDAO	`findByAyah(ayahId)`: the tafsir entries that explain one verse.
HadithDAO	`findById(id)`; `findApprovedByMasala(masalaId)` (approved HADITH links only).
QanunDAO	`findById(id)`; `findApprovedByMasala(masalaId)` (approved QANUN links only).
MadhabPositionDAO	`findByMasala(masalaId)`: the madhab positions of one issue.

> Every DAO receives the `Database` in its constructor. Because only the DAOs touch the data, we can later replace the in-memory lists with web APIs or a real database by editing the DAOs, and nothing else changes.

Application (2 classes)

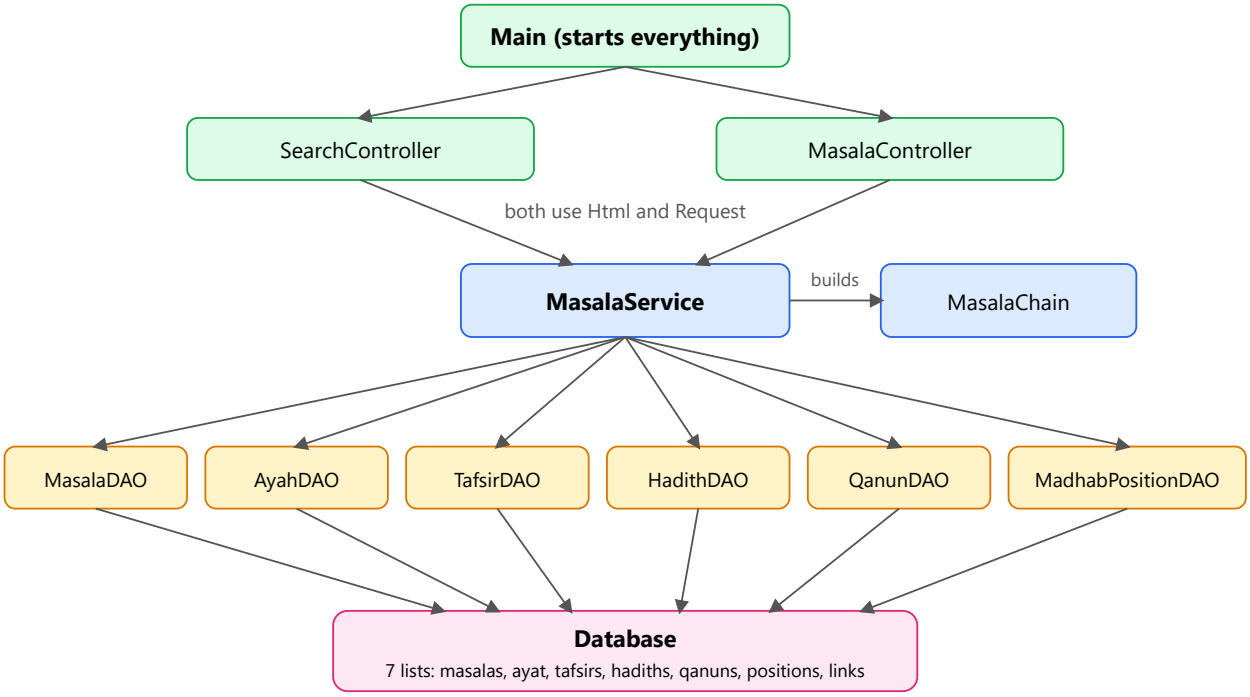
Class	What it does
MasalaService	The logic of the system. Receives the six DAOs in its constructor. Methods: `getAllMasalas()`, `searchByKeyword(text)` (FR-3), `getChapters(category)` and `getMasalasInChapter(category, chapter)` for browsing (FR-4), and `getChain(masalaId)` which builds the whole chain (returns null if the issue does not exist).
MasalaChain	A "parcel" holding everything about one issue: the issue, its verses, tafsirs, hadiths, positions and laws. Has `getTafsirsOf(ayahId)` to pick the tafsir of one verse. The page receives one object instead of six lists.

Presentation (5 classes)

Class	What it does
Main	Where the program starts. Builds all layers bottom-up (Database → DAOs → Service), creates the web server on port 8080 and registers the controllers: `/masala` to `MasalaController`, everything else to `SearchController`.
SearchController	Answers `/` (home page with search box and the Browse tree: Book → Chapter → Issue) and `/search?q=...` (results with count and time in ms, for NFR-1). Any other address gives a 404 page.

MasalaController	Answers `/masala?id=N`: the issue page with four sections (Qur'an and tafsir, hadith, four madhab positions side by side, qanun). Shows friendly messages for empty sections and a 404 for an unknown or non-numeric id.
Html	Helpers used by every page: `escape()` makes text safe (protects against injected scripts), `page()` wraps content in the common layout with CSS and the phone-friendly viewport tag (NFR-2), `send()` sends the page with a status code.
Request	`getQueryParam(exchange, name)`: reads a value from the address, for example `q` in `/search?q=wali`.

6. How the classes are linked



Arrow = "uses" (the class at the tail holds the class at the head and calls its methods)
The 7 domain classes are used by all of these and are not drawn to keep the picture readable.

How the wiring is done

Classes do not create the classes below them. `Main` creates everything and passes each object to the one that needs it (this is called **constructor injection**):

```
Database database = new Database();
MasalaDAO masalaDAO = new MasalaDAO(database);        // DAO receives the Database
...
MasalaService service = new MasalaService(masalaDAO, ayahDAO, ...);  // Service receives DAOs
server.createContext("/masala", new MasalaController(service));      // Controller receives
Service
```

Because each class receives what it needs, it is easy to replace one part (for example give the DAOs a different data source) without editing the others.

7. The data model: how records connect by id

Records are connected by **ids**, like student numbers. No object holds another object directly; it holds the other one's id.

Record	Connects to
Tafsir.**ayahId**	the Ayah it explains

MadhabPosition.**masalaId**	the Masala it belongs to
Link.**masalaId** + Link.**targetId** + Link.**type**	the Masala, and the Ayah, Hadith or Qanun that supports it (type says which list targetId belongs to)

Example: Link 5 says *issue 4 is supported by AYAH 3* (citation "Qur'an 2:229", status APPROVED). So for issue 4, `AyahDAO.findApprovedByMasala(4)` finds link 5 and returns Ayah 3 (Surah Al-Baqarah 2:229). `TafsirDAO.findByAyah(3)` then returns the tafsir entry of that verse.

> **The approval rule (FR-6, NFR-4):** a link has a status. Students only see links whose status is APPROVED, and every link has a citation. This is enforced inside the DAOs, so a pending or rejected link can never reach a student by accident.

8. Requirements coverage (as of today)

Req.	Meaning	Status
FR-1	Show the chain from an issue to verse, tafsir, hadith, madhabs, qanun	Done (issue page); real texts come from websites in Step 5
FR-2	Show the four madhab positions side by side	Done (CSS grid; for issues 1, 3 and 4)
FR-3	Keyword search	Done
FR-4	Browse Book → Chapter → Issue	Done
FR-5	Citation on every item	Done in the data model and displayed on the page
FR-6/7	Contributors propose links; reviewers approve or reject	Link status exists; login, propose and review pages are still to build
NFR-1	Search under 2 seconds	Done, the time in ms is shown on the results page
NFR-2	Works on phones	Done (viewport tag, stacking grid)
NFR-3	Passwords protected	Not started (login not built)
NFR-4	No link without a citation	Data model supports it; propose form must enforce it

9. Status, limits and what is left

Done

Layered skeleton, home page, search, browse tree, issue page with the whole chain, data model for verses, tafsir, hadith, law, madhab positions and links.

Still to build

#	Task	Notes

1	Load real texts from websites (Step 5)	Arabic verse text, tafsir text, hadith text and grade, fetched with `java.net.http.HttpClient` from public JSON sources and cached in a local file. Fills the placeholder lines using the setters.
2	User, login, roles	New domain class `User`, `UserDAO`, hashed passwords, login page and session.
3	Propose a link (contributor)	Form with a required citation; creates a Link with status PENDING. Needs a `LinkDAO`.
4	Review links (reviewer)	List of PENDING links, approve or reject buttons.
5	More content	Madhab positions for issues 2, 5 and 6, more issues, with sources.
6	README and documents	Update the README to this final structure; make the diagrams match the code.

Honest limits

> - The data is **in memory**: it is created at start-up and disappears when the program stops, so proposals and approvals would not survive a restart yet.
> - The madhab positions, book references, hadith numbers (Bukhari 5087 and Muslim 1472 especially) and the law article are **draft data**. They must be verified by the supervisor against the real books before being presented as correct.
> - The code uses single `Link` and text statuses (such as "APPROVED") where the design diagram shows separate link classes and a User hierarchy. This is a simplification; state it in the report or update the diagram.
> - There are no automated tests yet, and no protection against forged form submissions (CSRF) because there are no forms that change data yet.